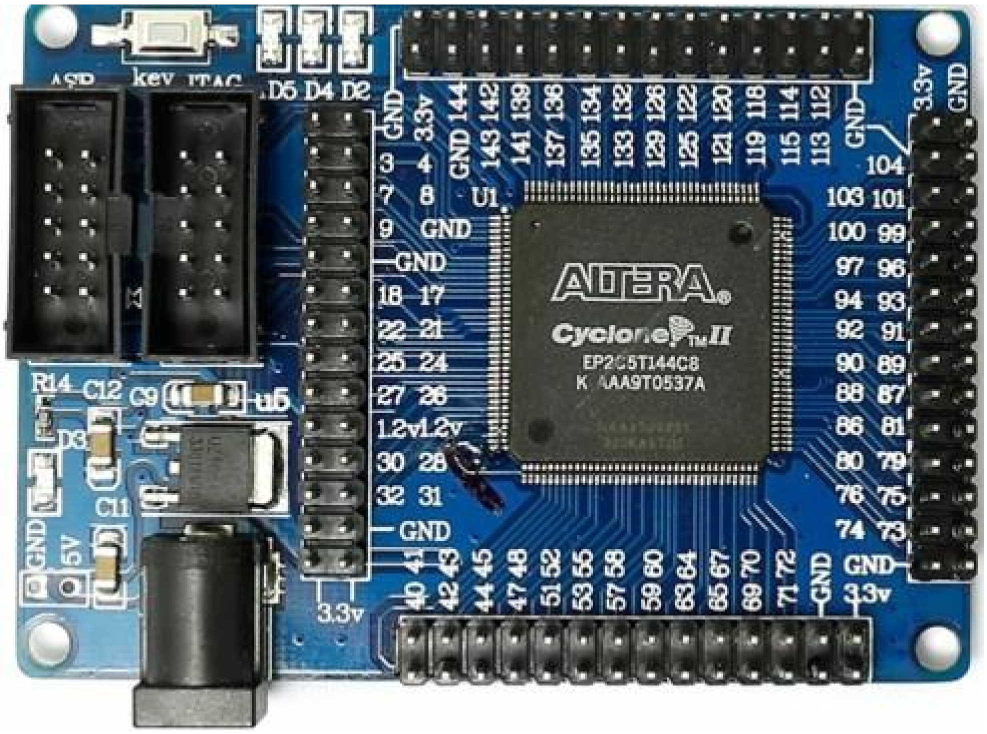
	Marwadi University **Faculty of Technology** **Department of Information and Communication Technology**
Altera Cyclone II FPGA Board	**Digital Systems on FPGA: Essential Design, Verification, and Implementation**
Standard Operating Procedure	**Date:** | **Enrollment No: 92301733025**

Experiment 1

Introduction to FPGA Architecture and Design Methodologies



Objective:

To provide an in-depth understanding of FPGA architecture, focusing specifically on the Altera Cyclone II EP2C5T144C8 FPGA. This experiment will cover the historical evolution of FPGAs, detailed architectural insights, and the complete design methodologies employed in FPGA-based digital design. Students will develop the technical expertise necessary for advanced FPGA design and implementation.

1. Introduction to FPGAs:

1.1. What is an FPGA?

- A Field-Programmable Gate Array (FPGA) is a semiconductor device based on a matrix of configurable logic blocks (CLBs) connected via programmable interconnects. FPGAs are reconfigurable, meaning that the logic functions within the FPGA can be reprogrammed to implement different designs or algorithms after manufacturing. This reconfigurability contrasts with fixed-function Application-Specific Integrated Circuits (ASICs), making FPGAs highly versatile for prototyping and deployment in a wide range of applications.

1.2. Evolution of FPGA Technology: From PLAs to

Modern FPGAs Origins of Programmable Logic:

- The foundation of FPGA technology began with Programmable Logic Arrays (PLAs) and Programmable Array Logic (PALs) in the 1970s. PLAs allowed for the implementation of combinational logic functions through programmable AND and OR gates, but they were limited in complexity. PALs simplified the design process by offering a fixed OR array and programmable AND array, making them more efficient and easier to use, though still restricted in scalability.

CPLDs as a Bridge to FPGAs:

- To handle more complex designs, Complex Programmable Logic Devices (CPLDs) were developed. CPLDs integrated multiple PAL-like blocks within a single chip, connected through a programmable interconnect matrix. This provided more logic capacity and flexibility than earlier PLDs but still fell short in handling very large-scale designs.

Birth of the FPGA:

- The limitations of PLAs, PALs, and CPLDs led to the development of Field-Programmable Gate Arrays (FPGAs) in the mid-1980s, with Xilinx introducing the first commercial FPGA in 1985. FPGAs introduced Configurable Logic

Blocks (CLBs), which contained logic elements, flip-flops, and interconnects that could be configured to implement any digital circuit. The programmable interconnects between CLBs allowed for high flexibility and scalability, making FPGAs suitable for complex designs.

Altera's Contributions:

- Altera, founded in 1983, became a major player in the FPGA industry. Their early innovations, like the EPROM-based EP300, laid the groundwork for the company's success. Altera's Cyclone series, particularly the Cyclone II, balanced cost, performance, and power efficiency, making it ideal for both educational and commercial use. The Cyclone II EP2C5T144C8 FPGA, with 4,608 Logic Elements, embedded memory blocks, and dedicated multipliers, exemplifies this balance, providing a robust platform for digital design.

1.3. Importance of FPGAs in Modern Electronics:

- FPGAs play a critical role in modern electronics, offering a platform for implementing complex digital circuits with a high degree of flexibility. In applications where time-to-market and the ability to update hardware post-deployment are crucial, FPGAs are invaluable. They are widely used in areas such as telecommunications, signal processing, automotive systems, aerospace, and even in the development of artificial intelligence and machine learning accelerators. FPGAs allow for parallel processing, real-time data manipulation, and high-throughput computation, making them essential in domains where performance and adaptability are paramount.

---

2. Detailed Overview of Altera Cyclone II EP2C5T144C8:

2.1. Architectural Overview:

- The Altera Cyclone II EP2C5T144C8 FPGA is a low-cost, high-performance device that strikes a balance between logic capacity, power consumption, and speed. This FPGA is built on a 90nm process technology, which contributes to its efficient power usage while providing a significant number of logic elements and other critical resources.

The Cyclone II series is designed to meet the needs of a broad range of applications, from simple logic designs to complex system-on-chip (SoC) implementations.

1. FPGA Chip (Center of the Board)

- **Cyclone II FPGA:** The central component of the board is the FPGA chip itself, labeled with pin numbers around it. The Cyclone II EP2C5T144C8 is a mid-range FPGA that includes a mix of logic elements, embedded memory blocks, and multipliers. These elements are organized into configurable logic blocks (CLBs) and are interconnected through a programmable routing fabric, allowing for the implementation of complex digital circuits.

2. Digital GPIO (General-Purpose Input/Output)

- **GPIO Headers (Top, Right, Bottom):** The board has several GPIO headers on the top, right, and bottom edges. These GPIO pins can be configured as inputs or outputs and are connected to the FPGA's internal logic. They are used for interfacing with external peripherals, sensors, actuators, or other electronic components. Each GPIO pin can be individually programmed to handle various digital signals, allowing for versatile interfacing options.

3. User LEDs and Power Indication Led (Top-Left Corner)

- **LED Indicators (D2, D3, D4, D5):** The board includes a set of user-programmable LEDs (D2, D4, D5), which can be used for debugging, status indication, or simple output visualization and a power indication Led (D3) that lights up when proper power is supplied to the board. These LEDs are connected to specific FPGA pins, allowing you to control them through your FPGA design. For instance, you might use these LEDs to indicate the status of certain logic conditions, signal activity, or to create visual feedback in response to inputs.

4. Programming Interfaces (Left Side)

- **JTAG and ASP Interfaces:**
  JTAG Interface: The board features a JTAG (Joint Test Action Group) interface, which is commonly used for programming, testing, and debugging FPGA designs. Through the JTAG interface, you can load your bitstream into the FPGA, observe internal signals, and perform in-circuit verification.
  ASP Interface: The ASP (Active Serial Programming) interface is another method of configuring the FPGA, typically used for programming the FPGA's configuration memory, allowing the design to be retained after power is cycled.

5. Power Supply (Bottom-Left Corner)

- **DC Power Jack (5V Input):** The board is powered through a DC power jack, supporting an input voltage maximum of 5V. This power input is regulated and distributed across the board to power the FPGA and other components. The board also features a 3.3V output pin, which is typically used to power external peripherals that are interfaced with the GPIO pins.

6. Ground Pins and 3.3V Pin

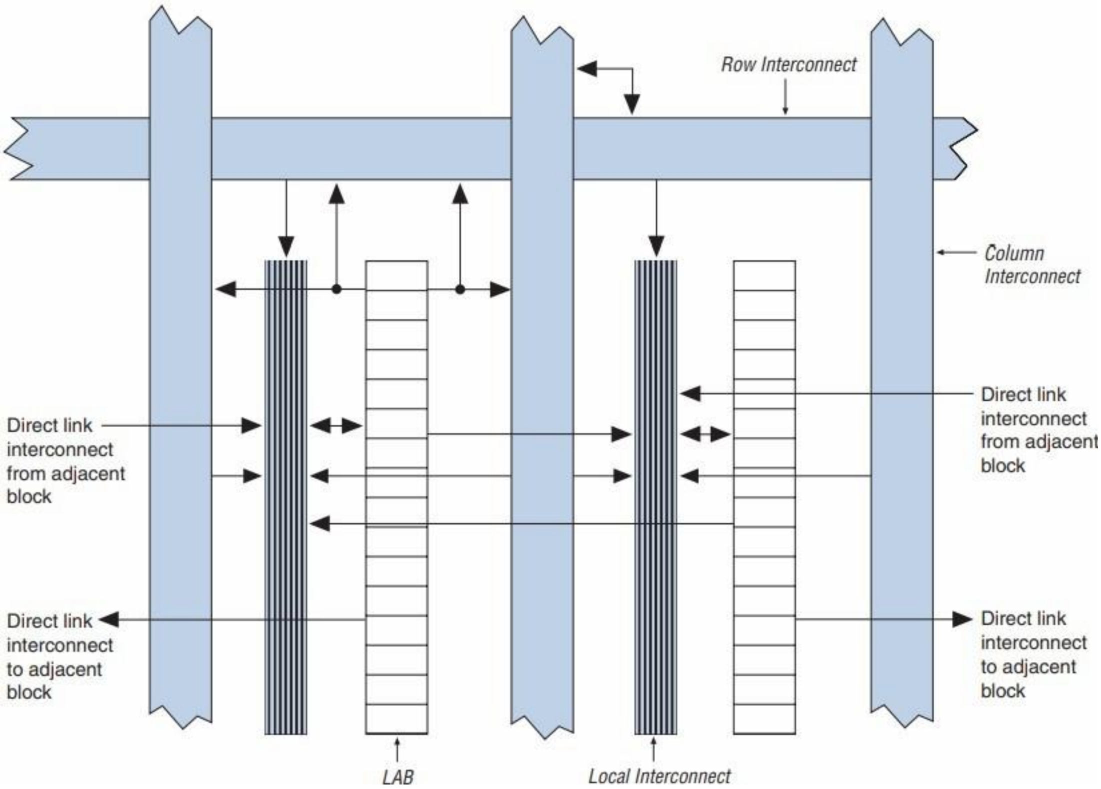
- **GND and 3.3V (Vcc) Pins:** There are multiple ground (GND) pins distributed across the board, providing a common reference point for the FPGA and any connected peripherals. The 3.3V (Vcc) pin provides a stable power source for external components, especially those connected to the GPIO headers.

7. Key Switch

- **Key Switch:** Located near the programming interfaces, this switch might be used for initiating programming mode or resetting the FPGA. It could also serve as a simple input for testing or controlling basic functions on the board.

2.2. Logic Array Block (LAB):

Figure 2–5. Cyclone II LAB Structure



- **Overview**:
  LABs are the fundamental building units within the Cyclone-II architecture. Each LAB contains a group of Logic Elements (LEs), interconnect resources, and control logic. The LABs are distributed in a grid pattern across the FPGA fabric, providing a structured and scalable way to implement complex digital circuits.
- **LAB Structure**:
  Each LAB in the Cyclone-II FPGA comprises **16 Logic Elements (LEs)**. These LEs are interconnected and share common control signals, such as clocks and resets.
  The LAB is designed to optimize both logic and routing efficiency, allowing the FPGA to achieve high performance with low power consumption.
- **LAB Interconnect**:
  LABs are connected to each other through a versatile and programmable

interconnect structure. This interconnect allows for flexible connections between

LABs and other blocks within the FPGA, such as memory blocks, DSP blocks, and I/O elements.

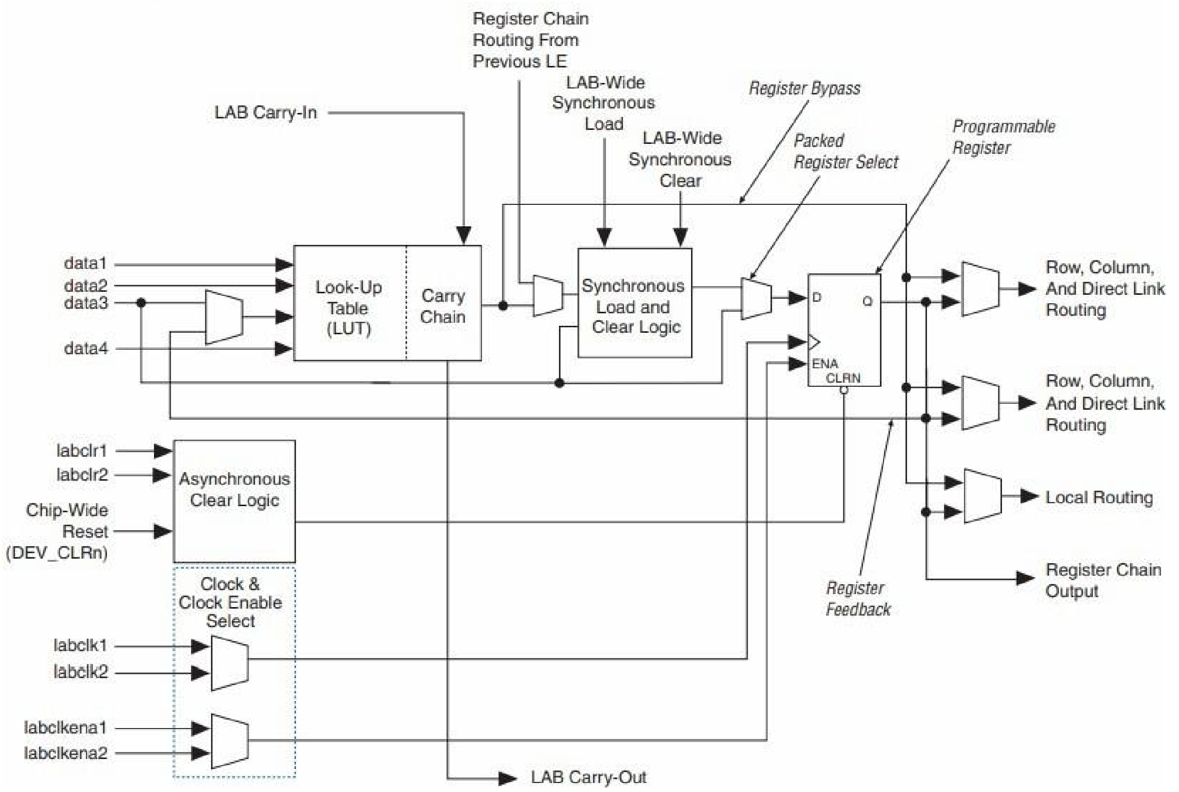
LABs have access to both local and global routing resources. Local routing is used for connections within or between adjacent LABs, while global routing handles connections across the FPGA, allowing for efficient implementation of wide logic functions or high-speed clock networks.

- **Control Signals**:

  LABs share control signals that manage the operation of the LEs within them. These include clock signals, synchronous and asynchronous reset signals, clock enables, and other control inputs that are essential for coordinating the timing and operation of the logic elements.

2.3. Logic Elements (LEs):

Figure 2–2. Cyclone II LE



- The core building block of the Cyclone II FPGA is the Logic Element (LE). Each LE is a small unit of logic that consists of a 4-input lookup table (LUT), a programmable flip-flop, and additional carry and cascade logic. The LUT can implement any Boolean function of up to four variables, while the flip-flop can be configured as a D flip-flop or a latch. The carry and cascade logic are used to implement arithmetic functions such as addition or to propagate signals across LEs efficiently.
- The Cyclone II EP2C5T144C8 contains 4,608 LEs, which can be organized and interconnected to perform complex combinational and sequential logic functions. The FPGA's architecture is optimized for efficient use of LEs, allowing designers to implement large designs without exhausting resources.

2.4. Embedded Memory Blocks:

- The EP2C5T144C8 includes 13,104 bits of embedded memory, divided into several M4K memory blocks. Each M4K block can be configured in various modes, including single-port, dual-port, and true dual-port RAM, supporting word widths from 1 to 36 bits. These memory blocks are crucial for implementing data storage, FIFOs, and other memory-intensive functions within the FPGA.
- In addition to user-configurable memory, the FPGA includes dedicated DSP blocks that can be used for high-speed arithmetic operations, such as multiply-accumulate (MAC) functions, which are essential in signal processing applications.

2.5. Clock Management and PLLs:

- Clock distribution and management are critical in FPGA designs to ensure that all parts of the circuit operate synchronously. The Cyclone II FPGA includes multiple phase-locked loops (PLLs) that can be used to generate and manage clock signals. These PLLs allow designers to adjust clock frequencies, generate phase-shifted clocks, and implement clock-domain crossing techniques to ensure reliable operation in designs with multiple clock domains.
- The FPGA also supports clock-enable signals, which can be used to gate the clock to specific parts of the design, reducing dynamic power consumption by disabling inactive circuits.

2.6. I/O Capabilities and Interfacing:

- The Cyclone II FPGA supports a wide range of I/O standards, including LVTTL, LVCMOS, PCI, SSTL, and HSTL. This flexibility allows the FPGA to interface with various external devices and components, such as sensors, displays, memory devices, and communication interfaces.

- The EP2C5T144C8 features 89 user I/O pins, which can be configured as input, output, or bidirectional. The I/O pins are grouped into banks, each capable of operating at different voltage levels, enabling mixed-signal interfacing with peripherals operating at different logic levels.

---

3. FPGA Design Methodologies:

3.1. FPGA Design Flow Overview:

- The FPGA design flow is a systematic process that transforms a high-level design specification into a functional configuration that can be loaded onto the FPGA. The design flow includes several key stages, each involving specific tools and methodologies:

  Specification: The first step in the FPGA design flow is defining the functionality, performance requirements, and constraints of the design. This specification serves as the blueprint for the entire project, detailing what the FPGA is expected to do and under what conditions it must operate.

  Design Entry: During this stage, the design is entered into the development environment using either a Hardware Description Language (HDL) such as Verilog or VHDL, or a schematic capture tool. HDLs are preferred for complex designs as they offer a high level of abstraction and reusability.

  Synthesis: The synthesis tool converts the HDL code into a gate-level netlist, which is a representation of the design in terms of logic gates and flip-flops. The synthesis process also optimizes the design for the target FPGA architecture, balancing factors such as area, speed, and power consumption.

  Simulation: Simulation is used to verify the functionality of the design before it is implemented on the FPGA. Functional simulation checks the logical correctness of the design, while timing simulation verifies that the design meets the required timing constraints. ModelSim is commonly used for this purpose, offering a detailed view of signal interactions and timing behavior.

  Implementation: This stage involves placing and routing the design onto the FPGA. The placement tool assigns logic elements, memory blocks, and I/O pins to specific locations on the FPGA, while the routing tool establishes the interconnections between these resources. The result is a fully routed design that can be analyzed for timing and resource utilization.

  Verification: After implementation, the design undergoes timing analysis to ensure that all paths meet the required timing constraints. Static timing analysis (STA) is

performed to check for timing violations such as setup and hold time errors. The design may also be tested in hardware using test benches or hardware-in-the-loop (HIL) techniques to ensure it performs correctly in real-world conditions.

Configuration: The final step is to generate the bitstream file, which contains the configuration data for the FPGA. This file is loaded onto the FPGA to configure the logic blocks, interconnects, and I/O settings according to the design. The FPGA is now ready to operate with the programmed functionality.

3.2. Tools and Resources:

- **Quartus II:** Altera's Quartus II software is the primary tool for FPGA design, providing a comprehensive suite of tools for design entry, synthesis, simulation, and implementation. Quartus II supports both Verilog and VHDL and includes features such as incremental compilation, design partitioning, and advanced timing analysis.
- **ModelSim:** ModelSim is a popular simulation tool used to verify the functional and timing behavior of FPGA designs. It supports mixed-language simulation (Verilog and VHDL) and offers a range of debugging features, including waveform viewing, signal tracing, and code coverage analysis.

---

4. Hands-on with Altera Cyclone II EP2C5T144C8:

4.1. Setting Up the Development Environment:

- **Hardware Setup:** Begin by connecting the Altera Cyclone II EP2C5T144C8 development board to your development PC using a USB-Blaster or equivalent programming cable. Ensure that all necessary drivers are installed, and that the board is recognized by the Quartus II software.
- **Software Installation:** Install the Quartus II software on your development PC. The software is available in various editions, with the Web Edition being free for educational purposes. Make sure to select the correct device family (Cyclone II) and device model (EP2C5T144C8) during project setup.
- **Project Creation:** Create a new project in Quartus II, specifying the EP2C5T144C8 as the target device. This project will serve as the workspace for all subsequent design, simulation, and implementation tasks. Set up the necessary design files, including Verilog or VHDL source files, constraint files (for pin assignments), and simulation testbenches.

4.2. Basic Design Implementation:

- **Hello World Example:** As an introductory exercise, implement a simple LED blink program to familiarize yourself with the FPGA design flow. The design will involve writing Verilog code to create a that toggles LEDs at an external input button.

Pin Planning:

For the push buttons, use the following pins:

- **Pin 125: Connected to i1**

For the LEDs, use the following pins:

- **Assign o[2] to Pin 134**
- **Assign o[1] to Pin 135**
- **Assign o[0] to Pin**

133 Verilog Code

Example:

```
module led_blink(input i1, output [2:0]o);

    not n1(o[2],i1);

    assign o[1] = i1;

    not n2(o[0],i1);

endmodule
```

This code creates a circuit consisting of two NOT gates and a wire. It has three outputs and one input. A combination of gate level and data flow modelling style is used for this design. It is a simple combinational circuit. The output is a vector (bus) which is three bits wide. The input is a scalar of one bit.

- **Synthesis and Implementation:** After writing the Verilog code, proceed with the synthesis and implementation stages. Quartus II will convert the Verilog code into a gate-level netlist, place and route the design onto the FPGA, and generate the necessary bitstream file for programming.

- **Programming the FPGA:** Use the Quartus II Programmer tool to load the bitstream file onto the FPGA. This step configures the FPGA according to the design, enabling the LED to blink as specified.
- **Verification and Debugging:** Verify the functionality of the design using the SignalTap II Logic Analyzer or external test equipment such as oscilloscopes or logic analyzers. Check the LED output for the expected blinking pattern, and debug any issues by examining internal signals and timing paths.

4.3 Output